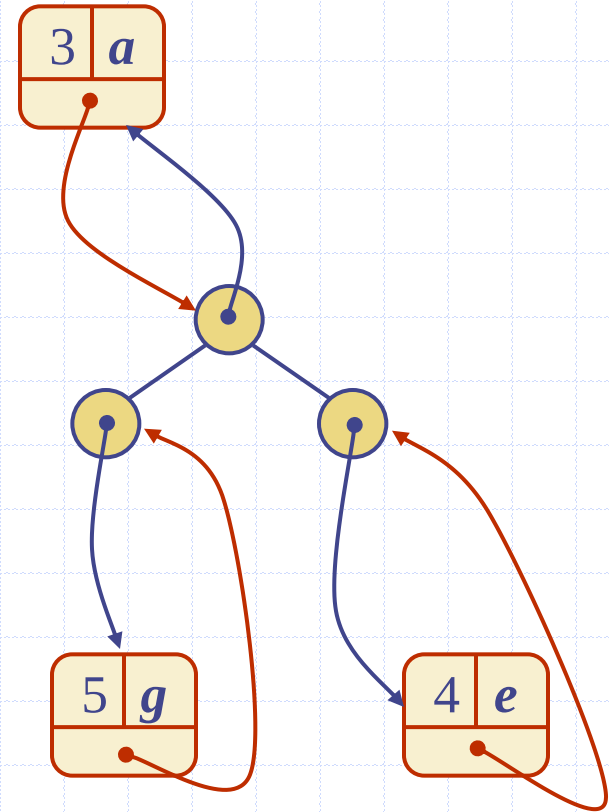


Adaptable Priority Queues



Items and Priority Queues

- An **item** stores a (key, value) pair
- Item fields:
 - **_key**: the key associated with this item
 - **_value**: the value paired with the key associated with this item
- Priority Queue ADT:
 - **add(k, x)**
inserts an item with key k and value x
 - **remove_min()**
removes and returns the item with smallest key
 - **min()**
returns, but does not remove, an item with smallest key
 - **len(P), is_empty()**



Example

- Online trading system where orders to purchase and sell a stock are stored in two priority queues (one for sell orders and one for buy orders)
(p, s) entries:
 - The key, p , of an order is the price
 - The value, s , for an entry is the number of shares



Example

- A buy order (p, s) is executed when a sell order (p', s') with price $p' \leq p$ is added (the execution is complete if $s' \geq s$)
- A sell order (p, s) is executed when a buy order (p', s') with price $p' \geq p$ is added (the execution is complete if $s' \geq s$)



Example

- What if someone wishes to cancel their order before it executes?
- What if someone wishes to update the price or number of shares for their order?

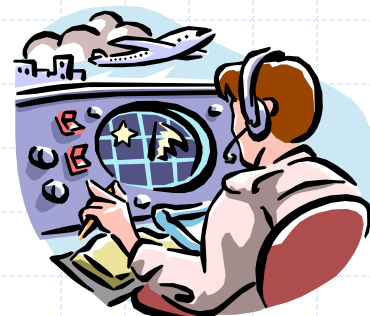
Methods of the Adaptable Priority Queue ADT

- ❑ **remove**(loc): Remove from P and return item e for locator loc.
- ❑ **update**(loc,k,v): Replace the key-value pair for locator, loc, with (k,v).

Locator vs. Position

- Intuitive notion:
 - Library:
 - ◆ Locator: Barcode
 - ◆ Position: Shelf, floor, ...
 - Airplane/train seat:
 - ◆ Locator: Seat number
 - ◆ Position: aisle, window, ...
 - Soccer:
 - ◆ Locator: Jersey number
 - ◆ Position: Goal post, midfield, ...



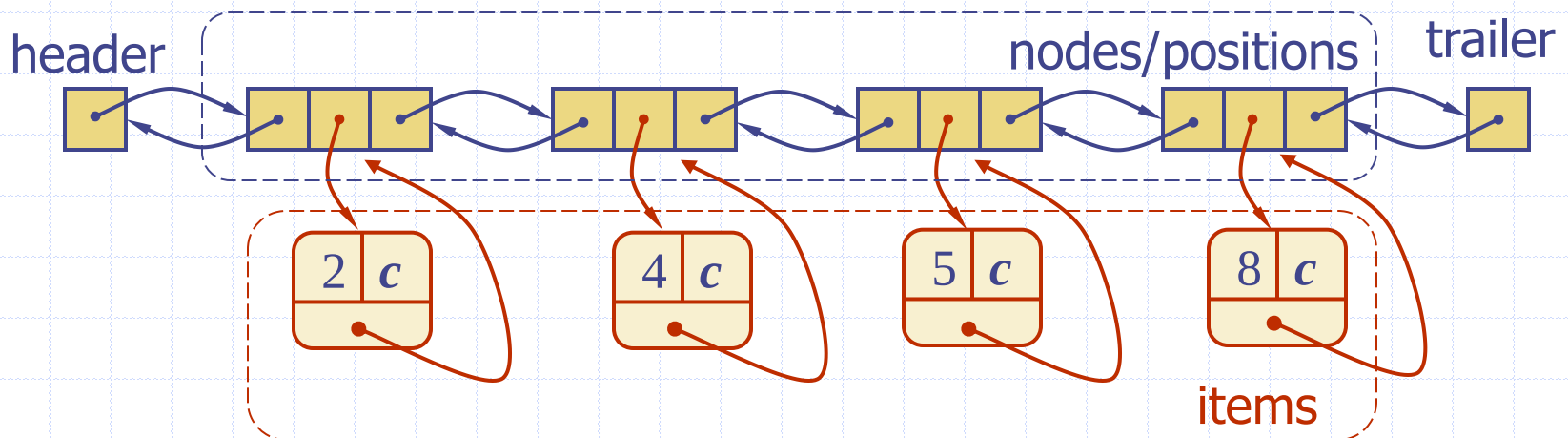


Locators

- A locator-aware item identifies and tracks the location of its (key, value) object within a data structure
- Intuitive notion:
 - Coat claim check
 - Valet claim ticket
 - Reservation number
- Main idea:
 - Since items are created and returned from the data structure itself, it can return location-aware items, thereby making future updates easier

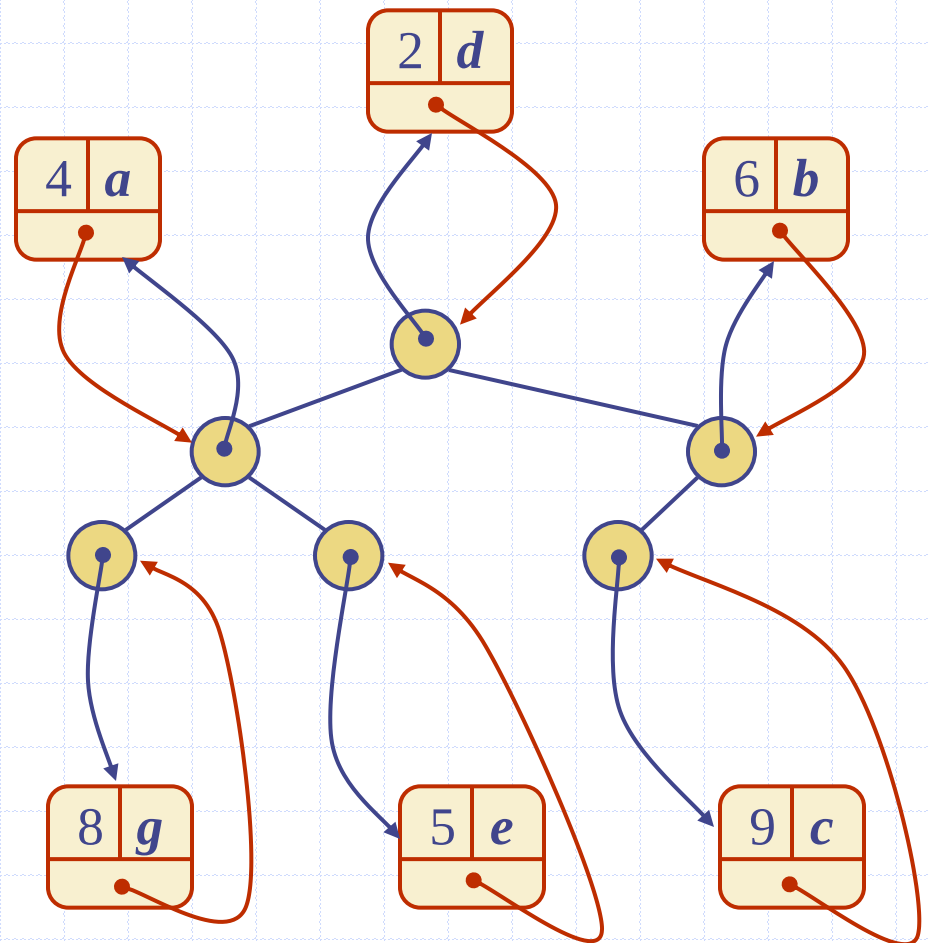
List Implementation

- A location-aware list item is an object storing
 - key
 - value
 - position (or rank) of the item in the list
- In turn, the position (or array cell) stores the entry
- Back pointers (or ranks) are updated during swaps



Heap Implementation

- ❑ A location-aware heap item is an object storing
 - key
 - value
 - position of the item in the underlying heap
- ❑ In turn, each heap position stores an item
- ❑ Back pointers are updated during item swaps



Python Implementation of Adaptable PQ

```
class AdaptableHeapPriorityQueue(HeapPriorityQueue):
    """A locator-based priority queue implemented with a binary heap."""

    #----- nested Locator class -----
    class Locator(HeapPriorityQueue._Item):
        """Token for locating an entry of the priority queue."""
        __slots__ = '_index'          # add index as additional field

        def __init__(self, k, v, j):
            super().__init__(k,v)
            self._index = j

    #----- nonpublic behaviors -----
    # override swap to record new indices
    def _swap(self, i, j):
        super()._swap(i,j)           # perform the swap
        self._data[i]._index = i     # reset locator index (post-swap)
        self._data[j]._index = j     # reset locator index (post-swap)

    def _bubble(self, j):
        if j > 0 and self._data[j] < self._data[self._parent(j)]:
            self._upheap(j)
        else:
            self._downheap(j)
```

Python Implementation of Adaptable PQ

```
#----- public behaviors -----
def add(self, key, value):
    """Add a key-value pair."""
    token = self.Locator(key, value, len(self._data)) # initiaize locator index
    self._data.append(token)
    self._upheap(len(self._data) - 1)
    return token

def update(self, loc, newkey, newval):
    """Update the key and value for the entry identified by Locator loc."""
    j = loc._index
    if not (0 <= j < len(self) and self._data[j] is loc):
        raise ValueError('Invalid locator')
    loc._key = newkey
    loc._value = newval
    self._bubble(j)

def remove(self, loc):
    """Remove and return the (k,v) pair identified by Locator loc."""
    j = loc._index
    if not (0 <= j < len(self) and self._data[j] is loc):
        raise ValueError('Invalid locator')
    if j == len(self) - 1:
        # item at last position
        self._data.pop()
        # just remove it
    else:
        self._swap(j, len(self)-1)
        # swap item to the last position
        self._data.pop()
        # remove it from the list
        self._bubble(j)
        # fix item displaced by the swap
    return (loc._key, loc._value)
```

Performance

len, is_empty	$O(1)$
add	$O(\log n)$
min	$O(1)$
remove_min	$O(\log n)$
remove	$O(\log n)$
update	$O(\log n)$